



AI Agent Thực Sự Là Gì?

Vượt Xa Autocomplete — Reasoning, Planning, Acting

Playbook SDD + ADD | Chương 9

Giới Thiệu Chương

Chín chương trước đã xây dựng toàn bộ framework SDD. Chương này lùi lại một bước và đặt câu hỏi nền tảng: "AI Agent" mà chúng ta đã dùng xuyên suốt cuốn sách này thực sự là gì? Không phải ở mức marketing, mà ở mức kiến trúc kỹ thuật.

Hiểu rõ cơ chế cho bạn khả năng dự đoán agent sẽ hành xử như thế nào, debug khi agent sai, thiết kế workflow phù hợp với năng lực thực sự của agent.

Inference-time Compute — Cốt lõi của Agent hiện đại Các model như Claude Sonnet, OpenAI o1 dành thời gian "nghĩ" (reasoning) trước khi ra câu trả lời. Đây gọi là Inference-time Compute — tính toán tại thời điểm suy luận. Thay vì output token ngay, model chạy nhiều vòng internal reasoning, tự phản biện, tự kiểm tra, rồi mới commit vào output cuối cùng.

📌 **Agent hiện đại không phải "fast answering machine" — là "slow thinking machine".**

Mục Lục

Nội Dung Chương 9

Phần	Chủ đề	Nội dung
9.1	Định nghĩa Agent: Perception → Reasoning → Action	Vòng lặp, Inference-time, So sánh
9.2	Kiến trúc Agentic Coding	Tool Layer, MCP, Extended Thinking, Checkpoint System
9.3	Agentic vs. Conversational vs. Autonomous	Ba mức Autonomy, Human-in-the-Loop
9.4	Demo: Xem Agent Làm Việc Thực Tế	Session walkthrough, Log Analysis, Internal Monologue
9.5	Giới hạn của Agent Hiện tại — Đừng Kỳ Vọng Quá	Token Burn, Loop Trapping, Model Dependency, Context Cliff, API Hallucination

Định Nghĩa Agent: Perception → Reasoning → Action

Perception → Reasoning → Action

Vòng lặp Plan → Execute → Observe → Adjust

Inference-time Compute — Agent nghĩ như thế nào?

So sánh: Autocomplete → Chat → Agent

Environmental Feedback — Sự khác biệt then chốt

Định Nghĩa Kỹ Thuật AI Agent

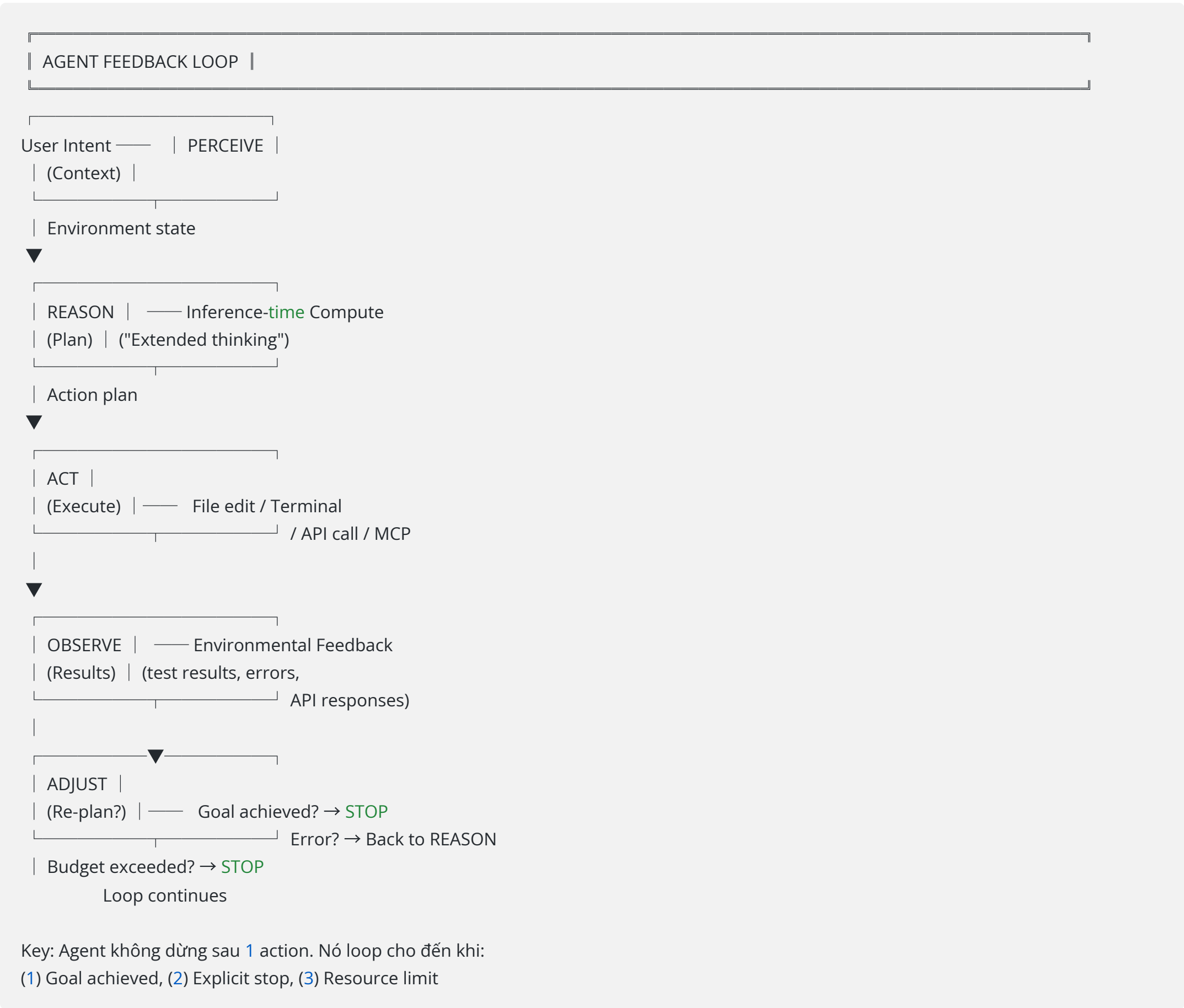
Từ "Agent" bị overloaded đến mức mất nghĩa. Để dùng từ này có ý nghĩa, cần định nghĩa chính xác: điều gì phân biệt một AI Agent thực sự với một chatbot thông thường?

Định nghĩa kỹ thuật

Một AI Agent là một hệ thống có khả năng nhận thức trạng thái môi trường (Perception), lập kế hoạch hành động dựa trên trạng thái đó (Reasoning), thực thi hành động trong môi trường (Action), và điều chỉnh kế hoạch dựa trên phản hồi từ môi trường (Feedback Loop) — lặp lại cho đến khi đạt mục tiêu.

📌 **Điểm quan trọng nhất: Environmental Feedback** Chatbot chỉ nhận feedback từ con người. Agent nhận feedback từ cả **môi trường** — lỗi terminal, kết quả test, response của API, output của command. Đây là sự khác biệt kiến trúc, không phải marketing.

Vòng Lặp Plan → Execute → Observe → Adjust



Inference-time Compute — Agent Nghĩ Như Thế Nào?

Thay vì trực tiếp generate output, model hiện đại dành compute để reasoning trước khi commit. Đây là lý do Claude Sonnet hay o1 "chậm hơn" GPT-3.5 — nhưng output có chất lượng cao hơn cho reasoning-intensive tasks.

Cơ chế	Tên	Hoạt động	Model điển hình	Use case tốt nhất
Fast answering	Standard generation	Token → token trực tiếp	GPT-3.5, Claude Haiku	Câu trả lời nhanh, simple tasks
Chain-of-thought	Step-by-step	Viết reasoning steps rồi conclude	GPT-4, Claude Sonnet	Multi-step math, logic
Extended thinking	Internal monologue	Thinking tokens ẩn trước output	Claude Sonnet (thinking mode)	Complex planning, code debugging
Search-augmented	Test-time search	Generate → verify → backtrack	o1, o3, Claude Opus	Formal verification, proofs

Extended Thinking trong Claude Khi bật "Extended Thinking", Claude generate các thinking tokens ẩn trước output. Bạn có thể thấy nội dung này trong API response dưới dạng blocks. Agent dùng thinking để: phân tích vấn đề, cân nhắc nhiều approach, phát hiện edge cases.

Cost: thinking tokens được charge như regular tokens — thường 2-5× nhiều hơn. Khi nào dùng: complex planning, debugging multi-file issues, architectural decisions.

So Sánh: Autocomplete → Chat → Agent

Không phải mọi AI coding tool đều như nhau. Ba thể hệ công cụ có kiến trúc và năng lực khác nhau căn bản:

Tiêu chí	Autocomplete (Copilot cũ)	Chat (ChatGPT/Claude.ai)	Agent (Cline/Claude Code)
Hành động	Suggest next tokens	Generate text/code block	Plan + execute + loop
Tự lập kế hoạch	✗ Không	✗ Không	✓ Có (multi-step plan)
Đọc codebase	⚠ Chỉ file hiện tại	⚠ Chỉ khi paste vào	✓ Tự đọc nhiều files
Thực thi lệnh	✗ Không	✗ Không	✓ Terminal, test runner
Environmental Feedback	✗ Không	✗ Chỉ từ user	✓ Errors, test results
Tự sửa lỗi	✗ Không	⚠ Khi user báo	✓ Tự detect + fix loop
Error Recovery	✗ Không có	✗ Không có	✓ Rollback + checkpoint
Context window	~2K tokens (current file)	8K–128K tokens	200K+ (via tools)

Environmental Feedback — Sự Khác Biệt Kiến Trúc Then Chốt

Đây là điểm phân biệt agent thực sự với mọi công cụ AI trước đó. Khi Cline chạy pytest và nhận được output:

```
$ pytest tests/cart/test_service.py -v
FAILED tests/cart/test_service.py::test_cart_merge_max_qty
AssertionError: assert 5 == 3
Full cart after merge has 5 items, expected max(3, 2) = 3
FAILED tests/cart/test_service.py::test_concurrent_add
IntegrityError: duplicate key value violates unique constraint "cart_items_cart_id_product_id_key"
1 passed, 2 failed in 0.23s
```

- ❏ **Agent không dừng lại và hỏi bạn** Agent đọc output, parse error messages, trace về code, identify root cause, generate fix, chạy lại test. Toàn bộ quá trình đó không cần human input trung gian. Đây là Environmental Feedback in action — sự khác biệt kiến trúc, không phải marketing.

9.2 — PHẦN HAI

Kiến Trúc Agentic Coding

Tool Layer, MCP, Extended Thinking, Checkpoint

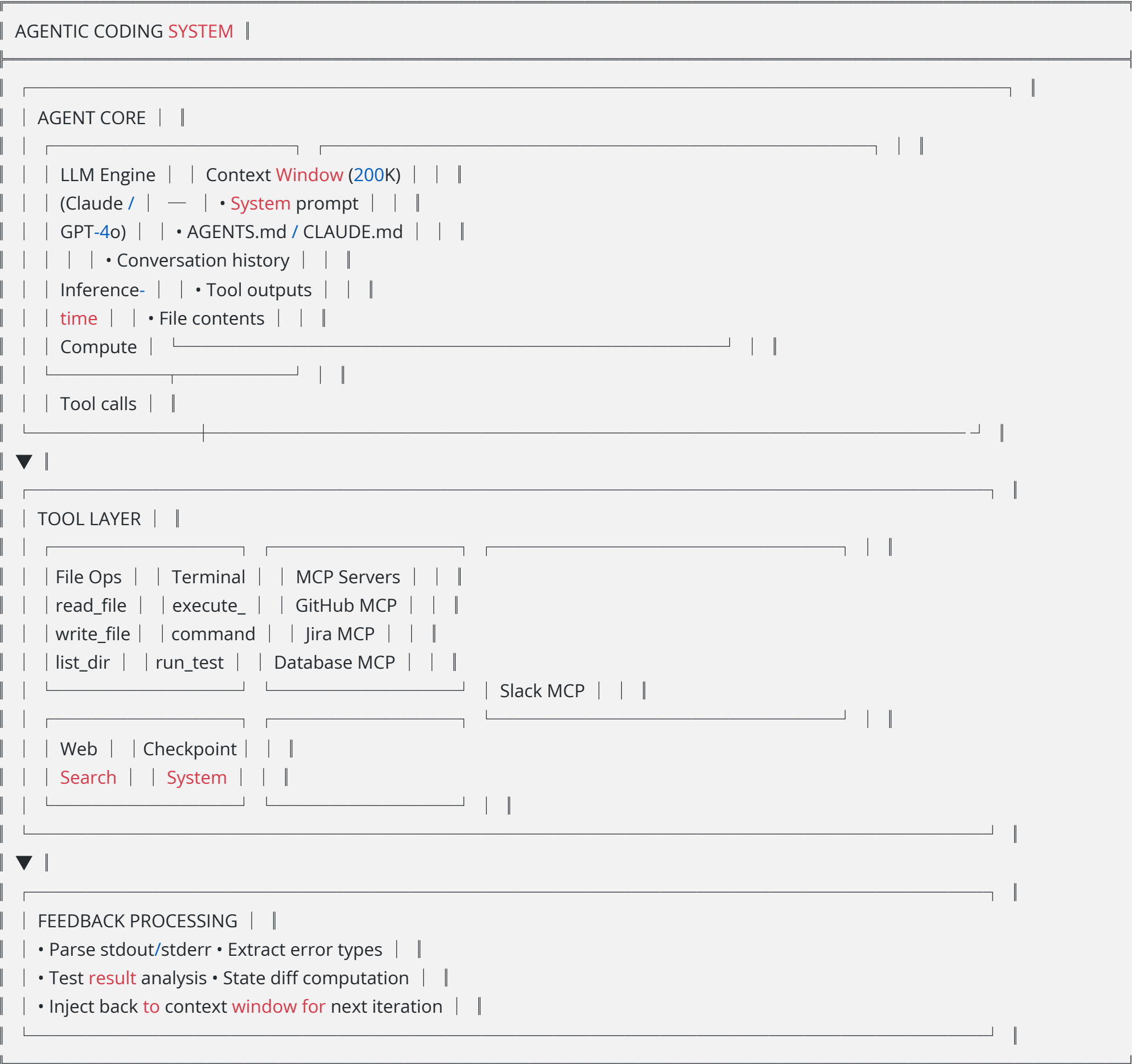
Sơ đồ kiến trúc tổng quan

Tool Layer — MCP như tiêu chuẩn ngành

Extended Thinking — Agent dừng lại và suy nghĩ

Checkpoint System — Cơ chế rollback khi agent sai

Sơ Đồ Kiến Trúc Tổng Quan



Tool Layer — MCP Như Tiêu Chuẩn Ngành

MCP (Model Context Protocol) chuẩn hóa giao tiếp: bất kỳ tool nào implement MCP protocol đều hoạt động với bất kỳ MCP-compatible host nào. MCP = "USB standard for AI tools".

Tool category	Capabilities	Ví dụ MCP Server	Khi agent dùng
File System	read, write, list, search	Filesystem MCP	Đọc codebase, edit files
Terminal	execute commands, capture output	Native (built-in)	Run tests, build, git, CLI
Web Search	query, fetch pages	Brave Search MCP	Tìm docs, error solutions
Version Control	git log, diff, blame, PR	GitHub MCP	Trace history, tạo PR
Project Mgmt	issues, tasks, sprints	Jira/Linear MCP	Đọc requirements
Database	query, schema inspect	PostgreSQL MCP	Verify data, check migrations
Communication	send messages, search	Slack MCP	Alert khi hoàn thành

📌 **MCP — Tại sao quan trọng cho SDD?** Trong SDD workflow, agent cần đọc spec từ GitHub Issues, đọc code từ filesystem, chạy tests qua terminal, và update Jira khi xong. Tất cả đều qua MCP. Xem Chương 4 để biết cách setup MCP trong Cline và Claude Code.

Extended Thinking — Agent "Dừng Lại và Suy Nghĩ"

Extended Thinking là cơ chế cho phép model thực hiện inference-time compute — dành token để "nghĩ" trước khi output. Khi agent nhận một task phức tạp, nó chạy một internal monologue dưới dạng blocks.

```
# Ví dụ: Agent nhận task "Fix the failing cart merge test"
```

```
# === THINKING (internal, thường ẩn) ===
```

```
<thinking>
```

Nhìn vào test failure:

AssertionError: assert 5 == 3

Full cart after merge has 5 items, expected $\max(3, 2) = 3$

Test expectation: merge của cart có 3 items và cart có 2 items
với cùng product → result phải là $\max(3, 2) = 3$ items (không phải 5)

Điều này suggest rằng current implementation đang ADD quantity
thay vì taking MAX. Cần kiểm tra CartMergeService.merge() method.

Approach 1: Fix merge logic → change add to $\max$

Approach 2: Check nếu spec thực sự muốn $\max$ (có thể test sai?)

Đọc SPEC.md §3: "merge rule: keep $\max(\text{guest_qty}, \text{user_qty})$ "

→ Test đúng, code sai. Fix CartMergeService.

Plan: T1: Write CartMergeService + T2: Write unit tests + T3: Run tests

```
</thinking>
```

```
# === OUTPUT (visible) ===
```

```
# "Tôi thấy vấn đề trong CartMergeService.merge().
```

```
# Đang cộng quantity thay vì lấy max. Sửa ngay."
```

Checkpoint System — Cơ Chế Rollback Khi Agent Sai

Tầng 1 — Git-based

Agent tạo git commit trước mỗi action thay đổi nhiều file. Rollback = git reset. Mạnh nhất, hoạt động cho mọi file change.

Tầng 2 — Tool snapshot

Trước khi overwrite file, agent lưu backup foo.py.bak. Rollback trong session hiện tại.

Tầng 3 — Session

Cline và Claude Code lưu toàn bộ conversation state. Có thể rewind về bất kỳ điểm nào.

```
# Checkpoint workflow trong thực tế (Cline built-in)
# Trước mỗi "dangerous action" (xóa, overwrite nhiều files):
AGENT: "Tôi sắp xóa 3 files và rewrite CartService.py.
Tôi đã tạo checkpoint tại commit abc123.
Để rollback: git checkout abc123"
[Awaiting approval: Yes/No]
```

```
# Khi agent detect nó đi sai hướng:
AGENT: "Test vẫn fail sau 2 lần sửa.
Rollback về checkpoint và thử approach khác."
→ git reset --hard abc123
→ Agent restart với fresh approach
```

```
# IMPORTANT: Agent tạo checkpoint trước, không sau.
# Rule: "checkpoint → action" not "action → checkpoint"
```

State Management — Agent Quản Lý Trạng Thái

Agent không có "memory" giữa các session (trừ khi dùng MCP Memory Server). Trong một session, state được quản lý qua context window — một "cuộn giấy" dài chứa toàn bộ lịch sử.

State type	Lưu ở đâu	TTL	Risk nếu mất
Working memory (current plan)	Context window	Session	Agent quên plan, replan sai
File changes	Filesystem	Permanent	Mất code nếu không checkpoint
Tool outputs	Context window (appended)	Session (truncated)	Agent "quên" test kết quả cũ
Conversation history	Context window	Session	Context coherence bị break
Cross-session memory	MCP Memory Server	Configurable	Agent không nhớ preferences

 **Context Window Full — Dấu hiệu nguy hiểm** Khi context window đầy (>150K tokens), agent bắt đầu "quên" phần đầu của task. Dấu hiệu: agent hỏi lại thông tin đã được cung cấp, re-plan từ đầu, ignore constraints. Giải pháp: summarize → clear → re-inject essentials. Prevention: break task thành subtasks <50K tokens per session.

Agentic vs. Conversational vs. Autonomous

Ba Mức Độ Autonomy — Kiến Trúc & Rủi Ro Khác Nhau

Ba mức độ Autonomy

**Bảng so sánh chi tiết + Error
Recovery**

**Human-in-the-Loop — Con
người là Gatekeeper**

Ba Mức Độ Autonomy

Tier 1: Conversational

H: "Viết function X" → AI: [generates code] → Human: "Fix bug trên dòng 5" → AI: [generates fix] [STOP — waits for human]

Pattern: **Human input → AI output → Human reviews** | Error recovery: NONE — human must spot and report

Tier 2: Agentic (Current sweet spot)

H: "Fix all cart tests" → AI: [reads files] → [identifies root cause] → [plans fixes] → [edits service.py]

AI: "Về to write to CartService.py. Approve?" → Human: [reviews diff] → Approve 

AI: [runs tests] → [2 fail] → [fixes + runs again] → "All tests green. Done."

Pattern: **Human intent → AI executes → Human gates** | Error recovery: BUILT-IN — AI self-corrects

Tier 3: Autonomous (Future / Restricted use today)

H: "Handle all Jira tickets labeled 'bug'" → AI: [polls Jira] → [reads tickets] → [reads code] → [fixes bugs] → [runs tests] → [creates PRs] → [merges when CI green] — no human approval

Pattern: **High-level goal → AI executes fully** | Error recovery: BUILT-IN + AUTOMATED rollback

Bảng So Sánh Chi Tiết + Error Recovery

Tiêu chí	Conversational	Agentic	Autonomous
Human input	Mỗi bước	Chỉ tại approval gates	Chỉ high-level goal
Tự lên kế hoạch	✗ Không	✓ Multi-step plan	✓ Long-horizon plan
Error Recovery	✗ Human phải báo	✓ Self-detect + fix	✓ Fully automated
Rủi ro khi sai	🟢 Thấp (human approve)	🟡 Vừa (gate controlled)	🔴 Cao (no human gate)
Traceability	⚠ Manual review only	✓ Log + checkpoint	⚠ Cần audit system
Cost efficiency	⚠ Human bottleneck	✓ 90% AI, 10% human	✓ Highest automation
Tool examples	ChatGPT, Claude.ai	Cline, Claude Code, Cursor	AI Software Engineers (2025+)
Best for	Simple tasks, quick questions	Feature dev, bug fixes	Maintenance, scheduled tasks

Human-in-the-Loop — Con Người Là Gatekeeper

Human-in-the-loop không phải bottleneck — đó là thiết kế cố ý. Agent làm 90% công việc nặng; con người chỉ can thiệp tại những điểm có rủi ro cao nhất.

Action	Risk level	Gate type	Lý do cần human
Read files, search web	Thấp	Auto-approve	Không thay đổi state
Write file (small edit)	Vừa	Show diff, 1-click approve	Reversible, cần eyeball check
Delete files	Cao	Explicit confirm + checkpoint	Hard to reverse
Run terminal commands	— Tùy lệnh	Show command, approve before run	Lệnh như rm -rf cần human xem
Git commit	Vừa	Show commit message + diff	Affects shared history
Deploy / Release	Cao	Full review + explicit approve	Production impact
Call external APIs	— Tùy API	Show request details	Cost, side effects, rate limits

Cline Config + Autonomy Spectrum

```
// .vscode/settings.json — Cline approval gates
{
  "cline.alwaysAllowReadOnly": true, // Read = auto-approve
  "cline.alwaysAllowWrite": false,  // Write = show diff
  "cline.alwaysAllowExecute": false, // Commands = explicit
  "cline.autoApproveEnabled": false, // No full auto
  // Allowlist cho specific safe commands
  "cline.allowedCommands": [
    "pytest",    // Run tests = safe
    "ruff",      // Linter = safe
    "mypy",      // Type check = safe
    "git status", // Read-only git = safe
    "git diff"
  ]
  // Anything NOT in allowlist = needs human approval
}
// Philosophy:
// Agent làm việc nhanh trên read/analyze/test.
// Human review tại "point of no return" moments.
// 10% approval time → 90% development quality assurance.
```

Vị trí các tool trên Autonomy Spectrum (2025) Conversational: ChatGPT, Claude.ai, Gemini web interface

Agentic: Cline, Claude Code, Cursor Composer, GitHub Copilot Workspace

Approaching Autonomous: Devin (Cognition AI), SWE-agent (Princeton)

Fully autonomous (no human) chỉ phù hợp cho isolated, reversible tasks.

Demo: Xem Agent Làm Việc Thực Tế

Session walkthrough — Fail, Self-correct, Internal Reasoning

Session walkthrough — 6 phases (kể cả khi fail)

Agent Log Analysis — Đọc log để debug

Internal Monologue — Tại sao Agent chọn A thay vì B

Session Walkthrough — Từ Prompt Đến Green Tests (Phase 1 & 2)

Task: Implement CartMergeService cho EcoShop (Case Study Ch.7). Spec đã approved. Agent yêu cầu implement + tests, không có additional guidance.

```
# === USER INPUT ===
Implement CartMergeService theo SPEC.md §3 merge rule.
Đọc code hiện có, viết implementation + tests.
Chạy tests sau khi xong.

# === PHASE 1: PERCEPTION — Agent reads codebase ===
→ list_directory("src/cart/")
# OUTPUT:
src/cart/service.py ← CartService (exists)
src/cart/repository.py ← CartRepository (exists)
src/cart/models.py ← Cart, CartItem models (exists)
# merge_service.py NOT YET EXISTS ← needs to create
→ read_file("src/cart/service.py")
→ read_file("src/cart/models.py")
→ read_file(".sdd/features/feat-cart-core/SPEC.md")

# === PHASE 2: REASONING — Agent lên plan ===
<thinking>
Merge logic cần handle:
1. Guest cart items + User cart items
2. Same product+variant: keep max(guest_qty, user_qty)
3. Guest-only items: add to user cart
4. User-only items: keep as is
5. After merge: delete guest cart

Edge cases từ SPEC clarification:
- product_id + variant_id = unique key (Q3 answer)
- Refresh snapshot_price after merge (Q1 answer)
- Concurrent merge: need transaction (Q2 from original spec)

Race condition: add FOR UPDATE lock in transaction.
Plan: T1: Write CartMergeService + T2: Write unit tests + T3: Run tests
</thinking>
```

Phase 3: Action — Agent Viết Code

```
# === AGENT ACTION: Create file ===  
→ write_file("src/cart/merge_service.py")  
  
class CartMergeService:  
    def __init__(self, repo: CartRepository, price_svc: PriceService):  
        self.repo = repo  
        self.price_svc = price_svc  
  
    async def merge(self, guest_cart_id: str, user_cart_id: str) -> Cart:  
        # EARS[Event]: WHEN Customer logs in with Guest cart  
        # EARS[Ubiquitous]: Run in transaction (race condition protection)  
        async with self.repo.transaction() as txn:  
            guest_cart = await txn.get_cart_for_update(guest_cart_id)  
            user_cart = await txn.get_cart_for_update(user_cart_id)  
  
            for g_item in guest_cart.items:  
                key = (g_item.product_id, g_item.variant_id)  
                u_item = user_cart.find_item(*key)  
  
                if u_item:  
                    # EARS[Ubiquitous]: Same product → keep max qty  
                    u_item.quantity = max(u_item.quantity, g_item.quantity)  
                    # EARS[Ubiquitous]: Refresh snapshot_price  
                    u_item.snapshot_price = await self.price_svc.current(*key)  
                else:  
                    # Guest-only item → add to user cart  
                    user_cart.add_item(g_item, refresh_price=True)  
  
            await txn.delete_cart(guest_cart_id)  
            return await txn.save_cart(user_cart)  
  
→ write_file("tests/cart/test_merge_service.py")  
# [writes 5 test cases for the 5 scenarios]  
→ execute_command("pytest tests/cart/test_merge_service.py -v")
```

Phase 4: Observe — Tests Fail! Không Phải Happy Path

```
# === TEST OUTPUT (environmental feedback) ===  
FAILED test_merge_guest_only_items  
AttributeError: Cart object has no attribute "add_item"  
self.repo returned Cart object, not expected CartProxy  
  
FAILED test_merge_max_qty_with_price_refresh  
RuntimeError: Cannot use transaction outside async context  
NullSession: transaction requires active session  
  
PASSED test_merge_user_only_items_unchanged  
PASSED test_merge_deletes_guest_cart  
2 passed, 2 failed in 0.31s  
# ← Agent NHẬN PHẢN HỒI TỪ MÔI TRƯỜNG  
# Không hỏi human. Tự phân tích.
```

📌 **Environmental Feedback in Action** Agent đọc output test, parse error messages, identify root cause. Toàn bộ quá trình không cần human input trung gian. Đây chính xác là điều phân biệt Tier 2 (Agentic) với Tier 1 (Conversational).

Phase 5: Reasoning — Agent Tự Debug

```
# === AGENT THINKING sau khi đọc failures ===
```

```
<thinking>
```

```
Failure 1: "Cart object has no attribute add_item"
```

- `add_item` không phải method của `Cart` model.
- Tôi assumed `Cart` có `add_item()` nhưng `models.py` không có.
- Fix: dùng `cart.items.append()` + create `CartItem` object directly.
- Check `models.py` lại để verify correct field names.

```
Failure 2: "Cannot use transaction outside async context"
```

- `self.repo.transaction()` không hoạt động trong unit test environment.
- Tests dùng mock repository không có real transaction support.
- Approach: Abstract transaction detail, mock trong tests.
- Hoặc: Đưa transaction lên caller (`CartService`), không trong `MergeService`.

```
Which approach?
```

```
SPEC nói CartService orchestrates everything.
```

```
PLAN.md: CartMergeService là helper, CartService owns transaction.
```

- Move transaction to `CartService.handle_login_merge()`.
- `CartMergeService` chỉ contain pure logic, easier to test.

```
Đây là architectural improvement, không chỉ bug fix.
```

```
Update both merge_service.py và service.py.
```

```
</thinking>
```

```
# AGENT: "Tôi thấy 2 vấn đề:
```

- # 1. `Cart.add_item()` không tồn tại — cần dùng `models` trực tiếp
- # 2. Transaction nên ở `CartService`, không `MergeService` (clean architecture)
- # Refactoring cả hai files."

Phase 6: Adjust — Fix và Re-test → Green!

```
# === AGENT ACTION: Fix based on reasoning ===
```

- `read_file("src/cart/models.py")` ← Verify Cart model fields
- `write_file("src/cart/merge_service.py")` ← Remove transaction, pure logic
- `write_file("src/cart/service.py")` ← Add transaction at CartService level
- `write_file("tests/cart/test_merge_service.py")` ← Update mocks

```
# === SECOND TEST RUN ===
```

```
→ execute_command("pytest tests/cart/test_merge_service.py -v")
```

```
PASSED test_merge_guest_only_items
```

```
PASSED test_merge_max_qty_with_price_refresh
```

```
PASSED test_merge_user_only_items_unchanged
```

```
PASSED test_merge_deletes_guest_cart
```

```
PASSED test_merge_handles_empty_guest_cart
```

```
5 passed, 0 failed in 0.18s
```

Agent Log Analysis — Đọc Log Để Debug

```
# Cline Agent Log — Location: View → Output → "Cline" dropdown
[10:23:01] TASK_START model: claude-sonnet-4-6 context_tokens: 15230
[10:23:02] TOOL_CALL: list_directory path: "src/cart/" tokens_added: 180
[10:23:03] TOOL_CALL: read_file path: "src/cart/service.py" tokens_added: 2250
[10:23:05] TOOL_CALL: read_file path: ".sdd/.../SPEC.md" tokens_added: 1800 ← Agent đọc spec!
[10:23:08] THINKING duration: 3.2s thinking_tokens: 850 ← Extended thinking enabled
[10:23:09] TOOL_CALL: write_file path: "merge_service.py" bytes: 1840 [AWAITING APPROVAL] ← Human gate
[10:23:15] HUMAN_APPROVED action: write_file
[10:23:18] TOOL_CALL: execute_command command: "pytest ... -v" exit_code: 1 ← Test fail! tokens_added: 420
[10:23:19] THINKING duration: 4.1s thinking_tokens: 1200 ← Longer think for error analysis
[10:23:25] TOOL_CALL: write_file (fix #1)
[10:23:28] TOOL_CALL: execute_command exit_code: 0 ← Green!
[10:23:30] TASK_COMPLETE total_duration: 89s total_tokens: 22450 (~$0.067)
      files_changed: 3 tool_calls: 12

# Total: 89s, $0.067 — for a task that would take dev 30+ min
```

Red Flags Trong Log — Dấu Hiệu Agent Đang Có Vấn Đề

Log pattern	Ý nghĩa	Action cần làm
Thinking duration > 10s liên tục	Agent stuck hoặc over-analyzing	Check nếu task quá rộng, chia nhỏ hơn
Cùng tool call lặp lại > 3 lần	Loop trap — fix này tạo ra lỗi khác	Interrupt, check root cause manually
context_tokens > 150K	Context window sắp đầy	Summarize và restart session
exit_code: 1 > 5 lần liên tiếp	Agent không thể fix test	Human review spec + code
TOOL_CALL không có THINKING trước	Agent acting without reasoning	Check nếu task quá simple hoặc agent confused
HUMAN_DENIED nhiều lần	Agent đề xuất risky actions	Review nếu task scope phù hợp

📖 **Đọc log = Debugging skill quan trọng nhất** Logs cho bạn biết chính xác agent đang làm gì, tại sao, và khi nào nó đi sai. exit_code, thinking duration, token count = những tín hiệu debugging chính yếu khi làm việc với agentic workflows.

Internal Monologue — Tại Sao Agent Chọn A Thay Vì B

Enable Extended Thinking để thấy lý do agent ra quyết định:

```
# Enable qua Claude API
response = client.messages.create(
    model="claude-sonnet-4-6",
    thinking={"type": "enabled", "budget_tokens": 5000},
    messages=[{"role": "user", "content": task}]
)

# Đọc thinking output
for block in response.content:
    if block.type == "thinking":
        print("AGENT THINKING:", block.thinking)

# === TYPICAL THINKING EXCERPT ===
# "Two approaches to implement this:
# A: Use Redis cache with 5min TTL
# B: Use PostgreSQL materialized view
#
# SPEC says: < 200ms p95 response time
# Current DB has 50K products → full scan too slow
#
# Redis: latency ~1ms, cost ~$20/month, cache invalidation needed
# Mat view: latency ~5ms, cost $0, auto-refresh by DB
#
# Both satisfy 200ms requirement.
# PLAN.md says: existing Redis cluster already deployed.
# → Use Redis (no new infrastructure, team already knows it)"

# Đây là chính xác lý do agent chọn Redis.
# Không phải "tôi thích Redis" — có reasoning chain rõ ràng.
```

Giới Hạn Của Agent Hiện Tại — Đừng Kỳ Vọng Quá

Đừng Kỳ Vọng Quá Mức — Hiểu để Design Đúng

**Token Burn — Chi phí ẩn của
Agentic Workflows**

**Loop Trapping — Khi Agent
"Ngáo"**

**Model Dependency —
Intelligence ceiling**

Context Window Cliff + API Hallucination

Tóm tắt giới hạn và cách đối phó

Token Burn — Chi Phí Ẩn Của Agentic Workflows

- ❏ **Kỳ vọng đúng = Kết quả tốt hơn** Agent là specialist cực giỏi trong phạm vi context window, nhưng có những điểm mù cơ bản. Design workflow tốt là "chơi đến điểm mạnh, tránh điểm yếu" — không phải "làm tất cả mọi thứ bằng agent".

Token budget breakdown — implement một feature

Bước 1: Initial context load (constant per session)

AGENTS.md + CLAUDE.md: 4,000 tokens

SPEC.md (current feature): 2,500 tokens

PLAN.md + TASKS.md: 1,500 tokens

Total setup: 8,000 tokens

Bước 2: Per-iteration costs (× nhiều lần)

Read source file (200 lines): 2,500 tokens × 5 files = 12,500

Thinking (Extended): 1,500 tokens × 3 rounds = 4,500

Generate code (150 lines): 2,000 tokens × 2 files = 4,000

Test output (50 lines): 600 tokens × 4 runs = 2,400

Error analysis thinking: 1,000 tokens × 2 errors = 2,000

Total dynamic: ~25,400 tokens

Total session cost:

Input tokens: ~28,000 × \$3/M = \$0.084

Output tokens: ~6,000 × \$15/M = \$0.090

Total: ~\$0.17 per feature task ← affordable!

Scale: 20 features/sprint × 2 devs × \$0.17 = \$6.80/sprint

Khi agent bị stuck và loop:

Normal task: 28,000 tokens

Loop task (10 iters): 280,000 tokens = \$1.70/task

5 tasks/day stuck: \$8.50/day = \$170/month extra!

Chiến Lược Tối Ưu Chi Phí Token

Kỹ thuật	Token saving	Trade-off
Dùng Haiku cho boilerplate tasks	70–85% cheaper	Lower quality for complex logic
Limit file reads (chỉ đọc relevant)	30–50%	Need good file organization
Summarize context trước khi bắt đầu	20–40%	Summary có thể miss details
Tắt Extended Thinking cho simple tasks	20–30%	Less reasoning quality
Set hard token limit per task	0% saving, prevent runaway	Task may not complete
Break large tasks thành atomic	15–25% per task	More session overhead

Budget control trong Cline

```
"cline.maxTokensPerTask": 50000 // hard stop at 50K
"cline.warnAt": 30000           // alert at 30K
```

Rule of thumb

- Normal feature: ~28K tokens = \$0.17
- Loop task: 10× = \$1.70
- Complex refactor: 50–200K tokens
- Break tasks <4h = <50K tokens

Loop Trapping — Khi Agent "Ngáo"

Loop Trapping xảy ra khi agent bị stuck trong một vòng lặp sửa lỗi: fix lỗi A tạo ra lỗi B, fix lỗi B tạo ra lỗi A (hoặc C). Agent không nhận ra nó đang loop vì mỗi iteration nhìn có vẻ "khác nhau".

Ví dụ Loop Trap thực tế Iteration 1: Fix TypeError → thêm null check.

Iteration 2: Null check tạo ra AssertionError (test expect non-null).

Iteration 3: Fix assertion → bỏ null check.

Iteration 4: TypeError lại. Agent ở iteration 1.

Vòng lặp tiếp tục, mỗi lần burn 3,000 tokens.

Nhận biết Loop Trap

- Cùng test fail > 3 lần liên tiếp
- Agent edit cùng một file > 5 lần
- Thinking time tăng mỗi iteration (agent confused hơn, không phải smart hơn)
- Agent bắt đầu thêm comments "Approach 5: trying a different..."
- Token counter tăng nhanh mà không có progress

Cách Xử Lý Khi Agent Bị Loop

- # Khi phát hiện agent đang loop — INTERRUPT ngay
- # 1. Dừng agent (Ctrl+C trong Cline)
- # 2. Đọc current state:
 - # - Test đang fail là gì?
 - # - Code hiện tại trông như thế nào?
 - # - Agent đã thử approach gì?
- # 3. Human analyze root cause
- # Thường loop xảy ra vì:
 - # a) Spec mơ hồ → AI không biết "đúng" là gì
 - # b) Architectural constraint chưa được spec
 - # c) Dependency issue → cần fix ở chỗ khác trước
- # 4. Provide clarification prompt, không phải "fix this"

- # SAI (tạo thêm loop):
 - # "Fix the test failure"

- # ĐÚNG (break loop với specific guidance):
 - # "Test fail vì cart.items.quantity không thể là 0.
 - # SPEC §5 Data: quantity là positive integer.
 - # Database constraint: quantity > 0 NOT NULL.
 - # Approach: kiểm tra quantity >= 1 trong CartService.add_item(),
 - # reject nếu < 1 với ValidationError.
 - # Dừng fix test — fix validation trong service."

- # 5. Nếu vẫn loop sau clarification → rollback + rewrite từ đầu
- # Rule of thumb: > 5 failed attempts = root cause issue,
- # không phải fixable incrementally.

Preventive Design — Tránh Loop Từ Đầu

5 nguyên tắc phòng tránh

- SPEC.md rõ ràng về constraints → agent không guess → ít loop
- Task atomic (<4h) → ít dependency → ít cascading failures
- Run Clarification Trigger TRƯỚC khi implement → catch ambiguity trước
- Set token limit per task → force interrupt sớm
- Provide patterns trong CLAUDE.md → agent biết "đúng cách"

Khi nào rollback vs. tiếp tục

- <3 attempts: tiếp tục, cung cấp guidance
- 3–5 attempts: human review spec + code
- 5 attempts: rollback + rewrite từ đầu
- Loop sau clarification: root cause issue

📌 Spec mơ hồ = loop trap đang chờ. Mỗi phút clarify spec trước implement tiết kiệm 10 phút debug loop.

Model Dependency — Intelligence Ceiling

Mọi agentic framework — Cline, Claude Code, Cursor Composer — đều là orchestration layer. Chất lượng output cuối cùng bị bounded bởi model LLM bên trong. Architecture agent tốt với model kém → kết quả tệ hơn architecture đơn giản với model tốt.

Model	Strengths cho coding	Weaknesses	Best for agentic tasks
Claude Sonnet	Code quality, reasoning, spec compliance	Cost > Haiku	Business logic, spec implementation
Claude Haiku	Speed, cost-efficient	Complex multi-step reasoning	Boilerplate, simple tasks
GPT-4o	Multi-modal, broad knowledge	Slightly less rigorous spec compliance	General coding, documentation
Llama 3 (local)	Privacy, no API cost	Weaker complex reasoning	Sensitive code, offline work
o1/o3 (OpenAI)	Deep reasoning, math	Very slow, very expensive	Algorithm design, formal verification

📌 **Implication cho SDD** Model tốt nhất cho SDD workflow là model có khả năng: (1) theo đúng EARS specification, (2) tự phát hiện khi đang vi phạm constraint, (3) đặt câu hỏi thay vì guess. Hiện tại Claude Sonnet thể hiện tốt nhất về cả ba tiêu chí này cho mid-complexity features.

Context Window Cliff + API Hallucination

Context Window Cliff — Khi Agent Đột Ngột "Quên"

Context window không fade out dần dần — nó có một "cliff" (vách đá). Khi đạt đến giới hạn (~180K tokens cho Claude), phần đầu của context bị drop. Output quality drops đột ngột.

- Symptom: agent generate code không tuân theo patterns đã thống nhất
- Symptom: agent hỏi lại thông tin đã có trong spec
- Fix: periodic context summarization ("Tóm tắt những gì đã làm và constraints quan trọng")

Hallucination về API/Library

Agent có thể confabulate API methods không tồn tại, đặc biệt với libraries ít phổ biến hoặc version mới.

Ví dụ hallucination:

Agent viết: `redis_client.set_with_ttl("key", value, ttl=300)`

Thực tế: `redis_client.setex("key", 300, value)`

Prevention:

1. Thêm vào CLAUDE.md: library version + usage examples

2. Cung cấp actual code examples trong spec

3. Sau khi agent write code: "Verify tất cả API calls là hợp lệ"

4. Luôn chạy code sau khi generate — import errors catch ngay

Long-range Dependency Blindness + Tóm Tắt Giới Hạn

Agent thường rất tốt với local code nhưng yếu hơn với long-range dependencies: thay đổi một file ảnh hưởng đến file khác 20 modules downstream. Đây là lý do integration tests quan trọng hơn unit tests khi dùng agent.

Giới hạn	Biểu hiện	Mitigation	Không thể tránh khi
Token Burn	Chi phí cao bất ngờ	Hard limits, task decomp, Haiku cho simple	Complex multi-file refactor
Loop Trap	Agent không converge	Clarification, rollback, human intervention	Spec ambiguous về constraints
Model Dependency	Quality ceiling	Use right model for task type	Budget restricts to weak model
Context Cliff	Agent "quên"	Periodic summarization, smaller tasks	Session >4h on complex codebase
API Hallucination	Code fails at runtime	Run code immediately, library docs in context	Obscure libraries, new versions
Long-range Blind	Hidden breaking changes	Run full test suite, not just unit tests	Large codebase, many dependencies

Tổng Kết Chương 9

Phần	Key concept	Điểm cốt lõi
9.1 — Định nghĩa	Perception→Reasoning→Action	Environmental feedback = sự khác biệt với chatbot
9.1 — Inference-time	Extended Thinking	Agent "nghĩ" trước khi làm — tốn token nhưng quality cao hơn
9.2 — Kiến trúc	Tool Layer + MCP	MCP chuẩn hóa agent tools — write once, use everywhere
9.2 — Checkpoint	State management	Checkpoint trước action — safety net cho rollback
9.3 — Autonomy	Three-tier spectrum	Agentic = sweet spot: AI làm 90%, human gates 10% risk
9.3 — HITL	Human-in-the-Loop	Gate is feature, not bug — control rủi ro cao
9.4 — Demo	Fail → Observe → Fix loop	Happy path ≠ realistic. Self-correct = real value
9.4 — Log analysis	Reading agent logs	exit_code, thinking duration, token count = debug signals
9.5 — Limits	Token burn + loop trap	Design workflow tránh điểm yếu, không fight against them